

AIQL ERP — 6-Month Development Plan

Hand to Sonnet one day at a time. Each day is a focused, self-contained deliverable.

- [AIQL ERP — 6-Month Day-by-Day Development Plan](#)
- [How to Use This Plan](#)
 - [The daily workflow](#)
 - [The Sonnet prompt pattern](#)
 - [Discipline rules](#)
 - [When a day spills over](#)
- [Phase Summary](#)
- [Phase 1 — V1 Launch Ready \(Days 1-20\)](#)
 - [Week 1: Foundation](#)
 - [Week 2: Trust & Export](#)
 - [Week 3: Authentication & Billing](#)
 - [Week 4: Differentiator + Polish](#)
- [Phase 2 — Bank Reconciliation \(Days 21-50\)](#)
 - [Weeks 5-6: PDF Parsers](#)
 - [Weeks 7-8: Matching Engine](#)
 - [Weeks 9-10: Workflow Integration](#)
- [Phase 3 — GSTN Auto-Reconciliation \(Days 51-90\)](#)
 - [Weeks 11-12: GSTN API Integration](#)
 - [Weeks 13-14: GSTR-2A/2B Pull](#)
 - [Weeks 15-16: Reconciliation Engine](#)
 - [Weeks 17-18: Workflow Integration](#)
- [Phase 4 — Tally Live Sync \(Days 91-130\)](#)
 - [Weeks 19-21: Desktop Agent](#)
 - [Weeks 22-23: Sync Pipeline](#)
 - [Weeks 24-25: Real-time Workflow](#)
 - [Week 26: Polish + Launch](#)
- [Templates & References](#)
 - [Sonnet prompt template \(use for any day\)](#)
 - [Weekly review checklist](#)
 - [When to deviate from the plan](#)
 - [Tracking sheet template](#)
- [Final Notes](#)
 - [What this plan delivers](#)
 - [What it doesn't deliver](#)
 - [The critical reminder](#)

AIQL ERP — 6-Month Day-by-Day Development Plan

Total: 130 working days across 4 phases **Pace assumption:** 5 productive engineering days/week, ~6-7 hours each **Used with:** Claude Sonnet for execution; this human reviews and approves

How to Use This Plan

The daily workflow

Each day's entry has 6 sections:

1. **Objective** — One sentence: what done looks like
2. **Files** — Specific paths to create or modify
3. **Implementation notes** — Architecture / approach guidance
4. **Tests** — What must pass before "done"
5. **Acceptance criteria** — Checklist for verification
6. **Sonnet prompt** — Copy-paste this into a fresh Sonnet session

The Sonnet prompt pattern

Each prompt includes:

- A "Read first" list of files Sonnet must read for context
- Constraints (file paths, test patterns)
- Acceptance criteria for verification
- Explicit instruction to run tests at the end

Discipline rules

- **One day = one focused deliverable.** If it doesn't fit a day, split across days.
- **Tests pass before "done."** No exceptions.
- **Each day stands alone.** Sonnet has no memory between sessions; the prompt must be self-contained.
- **Friday afternoon = review + plan next week.** Don't push features to weekends.

When a day spills over

Some days will. Realistic. Mark the day as "carried" and move the next day's task. Don't compress to maintain the schedule — quality matters more than pace.

Phase Summary

Phase	Days	Weeks	Calendar weeks (approx)	Pricing milestone
1: V1 Launch Ready	1-20	1-4	Weeks 1-4	₹999 / ₹2,999 / ₹6,999
2: Bank Reconciliation	21-50	5-10	Weeks 5-10	₹1,499 / ₹3,999 / ₹7,999
3: GSTN Auto-Reconciliation	51-90	11-18	Weeks 11-18	₹1,999 / ₹4,999 / ₹9,999
4: Tally Live Sync	91-130	19-26	Weeks 19-26	₹2,499 / ₹5,999 / ₹11,999

Phase 1 — V1 Launch Ready (Days 1-20)

Goal: Ship a product that can be handed to a paying customer.

Week 1: Foundation

Day 1 — Complete demo data loader UI

Objective: Verify "Load demo data" button works end-to-end, polish error handling.

Files:

- apps/web/src/components/value-summary-banner.tsx (already partially built)
- apps/web/src/app/api/v1/onboarding/load-demo/route.ts (already built)
- apps/web/src/lib/demo-loader.ts (already built)

Implementation notes:

- Demo loader already exists from previous session
- Verify it loads 3 sample companies (kumar_textiles, sharma_electronics, techvista)
- Add loading toast + success notification using existing UI patterns

- Add `unloadDemo` button on dashboard once demo is loaded

Tests:

- Manual: click button, see 3 connections appear within 30 seconds
- Add API integration test: `POST /api/v1/onboarding/load-demo` returns 200 with 3 connections

Acceptance criteria:

- ☐ Click "Try with sample data" → 3 demo connections created
- ☐ Page refreshes to show connections list populated
- ☐ Toast notification confirms success
- ☐ If clicked twice, doesn't duplicate (idempotent)
- ☐ All existing tests still pass

Sonnet prompt:

Read these files first:

- `apps/web/src/components/value-summary-banner.tsx`
- `apps/web/src/app/api/v1/onboarding/load-demo/route.ts`
- `apps/web/src/lib/demo-loader.ts`

Verify the demo loader works end-to-end:

1. Confirm the button in `value-summary-banner` triggers `POST` to `/api/v1/onboarding/load-demo`
2. Add a success toast notification after successful load
3. Add an "Unload demo data" button that appears once demo connections exist
4. Wire it to a new endpoint `/api/v1/onboarding/unload-demo` (function already exists in `demo-loader.ts`)
5. Add an integration test for both load and unload endpoints in `apps/web/src/__tests__`
6. Run ``pnpm --filter web test`` and ensure all pass

Day 2 — Onboarding wizard: Step 1 (Welcome)

Objective: First screen a new user sees after signup. Two-option choice: load demo or upload your own.

Files:

- `apps/web/src/app/(dashboard)/onboarding/page.tsx` (new)
- `apps/web/src/components/onboarding/welcome-step.tsx` (new)
- `apps/web/src/app/(dashboard)/page.tsx` (redirect if onboarding not complete)
- `packages/db/prisma/schema.prisma` (User model already has `onboardingComplete`)

Implementation notes:

- New route `/onboarding` that's the landing place after first signup
- Two big buttons: "Try with demo data" (uses loader from Day 1) and "Upload my own data"
- Sets `onboardingComplete=true` when either path completes
- Existing dashboard redirects to `/onboarding` if `!user.onboardingComplete`

Tests:

- Component test for welcome-step rendering
- E2E flow: new user → `/onboarding` → click demo → `/dashboard` with data

Acceptance criteria:

- ☐ New signup automatically routed to `/onboarding`
- ☐ Two clear options shown
- ☐ Choosing demo loads sample data and goes to dashboard
- ☐ Choosing upload goes to `/connections/new`

Sonnet prompt:

Read:

- `apps/web/src/app/(dashboard)/page.tsx`
- `apps/web/src/lib/auth.ts`
- `packages/db/prisma/schema.prisma` (look at User model, note `onboardingComplete` field)

Build onboarding wizard step 1:

1. Create `apps/web/src/app/(dashboard)/onboarding/page.tsx` – protected page for first-time users
2. Create `apps/web/src/components/onboarding/welcome-step.tsx` with two CTAs:
 - "Try with sample data" → calls `/api/v1/onboarding/load-demo` → redirects to `/`
 - "Upload my data" → `router.push('/connections/new')`
3. Update `apps/web/src/app/(dashboard)/page.tsx`: if `!user.onboardingComplete`, redirect to `/onboarding`
4. Create POST `/api/v1/onboarding/complete` endpoint that sets `onboardingComplete=true`
5. Call `/api/v1/onboarding/complete` after either CTA succeeds
6. Add a Skip link for users who want to explore without onboarding
7. Use the existing UI patterns (Button, Card components)
8. Run all tests

Day 3 — Delete connection from UI

Objective: User can remove a connection (and its data) from settings.

Files:

- `apps/web/src/app/(dashboard)/connections/[id]/page.tsx` (modify)
- `apps/web/src/app/api/internal/connections/[id]/route.ts` (already has DELETE)
- New: `apps/web/src/components/connections/delete-connection-dialog.tsx`

Implementation notes:

- DELETE endpoint already exists — just need UI
- Use confirmation dialog (require typing the connection name)
- On success: drop the upload table, delete UploadedFile + ErpConnection cascade
- Redirect to /connections after deletion

Tests:

- Test that DELETE cascades properly
- Test that the upload table is dropped
- Test confirmation dialog requires correct typing

Acceptance criteria:

- ☐ "Delete connection" button visible on connection page (with red accent)
- ☐ Confirmation dialog requires typing connection name
- ☐ On confirm: upload table dropped, all data removed
- ☐ User redirected to /connections with toast

Sonnet prompt:

Read:

- apps/web/src/app/(dashboard)/connections/[id]/page.tsx
- apps/web/src/app/api/internal/connections/[id]/route.ts
- packages/erp-connectors/src/file-upload/table-creator.ts (dropTempTable function)

Build the "Delete connection" UI:

1. Add a "Danger Zone" section at the bottom of /connections/[id] page
2. Create apps/web/src/components/connections/delete-connection-dialog.tsx using shadcn/ui Dialog
3. Dialog must:
 - Require typing the exact connection display name
 - Show warning text about permanent data loss
 - Disable the confirm button until name matches
4. On confirm: call DELETE /api/internal/connections/[id]
5. Make sure the existing DELETE endpoint also drops the upload_ table via dropTempTable
6. Redirect to /connections with a success toast after deletion
7. Add a test in apps/web/src/__tests__/connection-delete.test.ts
8. Run pnpm test

Day 4 — Privacy Policy + Terms of Service pages

Objective: Legal pages required for DPDP Act compliance.

Files:

- `apps/web/src/app/privacy/page.tsx` (new)
- `apps/web/src/app/terms/page.tsx` (new)
- `apps/web/src/app/layout.tsx` (footer with links)

Implementation notes:

- Use the template content provided in this prompt below
- Make pages SSR-renderable (no auth required)
- Add footer to main layout with these links
- Don't claim certifications you don't have

Tests:

- Pages render at `/privacy` and `/terms` without auth
- Footer links work from any page

Acceptance criteria:

- ☐ `/privacy` renders standalone (no login)
- ☐ `/terms` renders standalone (no login)
- ☐ Both linked from app footer
- ☐ Content covers: data collection, storage, sharing, deletion, contact

Sonnet prompt:

Read:

- `apps/web/src/app/layout.tsx`
- `packages/db/prisma/schema.prisma` (to understand what data is collected)

Build Privacy Policy and Terms of Service:

1. Create `apps/web/src/app/privacy/page.tsx` – comprehensive DPDP-compliant privacy policy:
 - What data we collect (signup info, uploaded GL data, query history)
 - How it's stored (AWS RDS Mumbai, encryption at rest)
 - PII handling (tokenisation before LLM calls)
 - Data retention (customer-controlled)
 - User rights (export, delete, audit)
 - Contact: `privacy@aiql.com`
 2. Create `apps/web/src/app/terms/page.tsx` – standard SaaS ToS:
 - Service description, account responsibilities
 - Pricing terms, refund policy (none for partial months)
 - Acceptable use, prohibited uses
 - Liability limitations (capped to fees paid)
 - Termination, governing law (Indian jurisdiction)
 3. Update `apps/web/src/app/layout.tsx` to add a footer with: Privacy | Terms | Security | Contact
 4. Run `pnpm typecheck`
-

Day 5 — Pricing page

Objective: Public pricing page at `/pricing` . Three tiers + pilot program callout.

Files:

- `apps/web/src/app/pricing/page.tsx` (new)
- `apps/web/src/components/pricing/pricing-cards.tsx` (new)
- `apps/web/src/lib/plans.ts` (centralised plan definitions)

Implementation notes:

- Three tiers: Starter ₹999, Professional ₹2,999, Business ₹6,999
- Annual toggle (2 months free)
- Pilot callout: "First 50 customers: 50% off for 3 months"
- CTAs: "Start free trial" → `/signup`, "Talk to sales" → `mailto`

Tests:

- Pricing page renders SSR
- Toggle annual/monthly updates prices

Acceptance criteria:

- ☐ `/pricing` renders without auth
- ☐ 3 tier cards with features clearly listed
- ☐ Annual/monthly toggle works
- ☐ Pilot program banner prominent
- ☐ Footer FAQ section with 5-7 common questions

Sonnet prompt:

Read:

- `apps/web/src/app/layout.tsx`
- `apps/web/src/components/ui/*` (available UI primitives)

Build `/pricing` page:

1. Create `apps/web/src/lib/plans.ts`:
 - Export `PLANS = [{ id, name, monthlyPrice, annualPrice, features, recommended }]`
 - 3 plans: Starter ₹999, Professional ₹2,999, Business ₹6,999
 - Annual prices = `monthly × 10` (2 months free)
2. Create `apps/web/src/components/pricing/pricing-cards.tsx` – 3-column grid with feature lists
3. Create `apps/web/src/app/pricing/page.tsx` with:
 - Hero: "Simple pricing. No surprises."
 - Toggle: Monthly | Annual (save 17%)
 - 3 pricing cards
 - Pilot banner: "First 50 customers get 50% off for 3 months"

- FAQ section (collapsible)
 - Final CTA: "Start your free trial" → /signup
4. Add link to /pricing in main nav and footer
 5. Run pnpm typecheck and pnpm --filter web test
-

Week 2: Trust & Export

Day 6 — PDF export for findings

Objective: "Export findings as PDF" button on close period / connection scan page.

Files:

- New: apps/web/src/lib/pdf-export.ts
- New: apps/web/src/app/api/v1/exports/findings-pdf/route.ts
- Modify: scan results UI to add button

Implementation notes:

- Use @react-pdf/renderer or generate HTML → use server-side puppeteer / playwright
- Cleanest: use pdfkit for programmatic PDF generation
- PDF includes: company name, scan date, summary stats, table of findings with severity colour coding

Tests:

- Unit test for PDF generation function (returns Buffer)
- Integration test for the export endpoint

Acceptance criteria:

- ☐ "Export PDF" button on scan results page
- ☐ Generated PDF includes all findings with ₹ amounts
- ☐ PDF has company name, period, generated-at timestamp
- ☐ Downloads as aiql-findings-{company}-{date}.pdf

Sonnet prompt:

Read:

- apps/web/src/app/(dashboard)/connections/[id]/scan/page.tsx
- packages/close-engine/src/scanner.ts (for ScanResult type)

Build PDF export for findings:

1. Install: pnpm add pdfkit @types/pdfkit --filter web
2. Create apps/web/src/lib/pdf-export.ts:
 - Function generateFindingsPdf(scanResult, connection): Promise<Buffer>

- Header: AIQL logo (text for now), company name, scan period
 - Summary stats: total findings, total ₹ exposure, severity counts
 - Findings table grouped by severity (critical/review/info)
 - Each finding: title, affected rows, ₹ exposure, top 3 examples
 - Footer: generated-at, "AIQL ERP – aiql.io"
3. Create `apps/web/src/app/api/v1/exports/findings-pdf/route.ts`:
 - POST endpoint takes `{ connectionId, periodId? }`
 - Auth check, ownership check
 - Run scan if no recent one, else use latest
 - Stream PDF response with proper Content-Type and filename headers
 4. Add "Export PDF" button on the scan results page
 5. Add a unit test in `packages/close-engine` or `apps/web` tests
 6. Test end-to-end via curl: download a PDF for one of the `validate_test` connections
-

Day 7 — Excel export for findings

Objective: Same data as PDF but as Excel (.xlsx) for analysis.

Files:

- New: `apps/web/src/lib/excel-export.ts`
- New: `apps/web/src/app/api/v1/exports/findings-xlsx/route.ts`

Implementation notes:

- Use `exceljs` library
- Multiple sheets: Summary, By Severity, All Findings (with filterable columns)
- Format ₹ amounts as currency, dates as dates

Tests:

- Unit test that returns a valid xlsx buffer
- Test that sheets contain expected data

Acceptance criteria:

- ☐ "Export Excel" button alongside PDF button
- ☐ xlsx opens cleanly in Excel and Google Sheets
- ☐ Columns are properly formatted (₹ as currency, dates as dates)
- ☐ Filterable header row enabled

Sonnet prompt:

Read:

- `apps/web/src/lib/pdf-export.ts` (from Day 6)
- `apps/web/src/app/api/v1/exports/findings-pdf/route.ts`

Build Excel export:

1. Install: `pnpm add exceljs --filter web`
 2. Create `apps/web/src/lib/excel-export.ts`:
 - Function `generateFindingsXlsx(scanResult, connection): Promise<Buffer>`
 - Sheet 1: Summary (counts by severity, total ₹)
 - Sheet 2: All Findings (columns: code, severity, title, rows, exposure, action)
 - Sheet 3: Top Issues (top 10 by exposure)
 - Use exceljs cell formats: currency for ₹, date for timestamps
 - Enable filter on header row
 - Freeze first row
 3. Create `apps/web/src/app/api/v1/exports/findings-xlsx/route.ts` (mirror PDF endpoint)
 4. Add "Export Excel" button next to "Export PDF" on scan results page
 5. Unit test that opening the file returns expected sheets/cells
-

Day 8 — Security one-pager at `/security`

Objective: Public page explaining security posture. Linked from footer.

Files:

- New: `apps/web/src/app/security/page.tsx`

Implementation notes:

- Cover: encryption at rest, in transit, PII tokenisation, data residency, retention, deletion rights, contact
- Don't claim certifications you don't have (no SOC 2, no ISO yet)
- Honest current state with roadmap items marked "in progress"

Tests:

- Page renders SSR without auth
- Linked from footer

Acceptance criteria:

- ☐ `/security` renders cleanly
- ☐ All 8 sections present
- ☐ No false claims (no SOC 2 yet, no ISO yet)
- ☐ Contact:  security@aiql.com

Sonnet prompt:

Build `/security` page with these sections:

1. Data residency – AWS RDS ap-south-1 (Mumbai)
2. Encryption at rest – AES-256 via AWS KMS

3. Encryption in transit – TLS 1.3
4. PII handling – tokenised before LLM calls, never logged in plaintext, never sent to third-party AI providers raw
5. Authentication – Lucia + argon2 password hashing, optional 2FA (planned Q3 2026)
6. Access controls – role-based, audit logged
7. Data retention – customer-controlled, default 365 days, deletion within 24 hours of request
8. Compliance posture – DPDP Act aligned (current), ISO 27001 (in progress Q3 2026), SOC 2 Type II (planned Q1 2027)
9. Incident response – disclosure within 24 hours per DPDP Act
10. Contact – security@aiql.com

Use clean visual hierarchy: section headers, icons (lucide-react), short paragraphs. No tables.

Add to footer alongside Privacy / Terms / Contact.

Day 9 — Sentry error monitoring

Objective: Server + client errors logged to Sentry. You see them before customers report.

Files:

- New: apps/web/sentry.client.config.ts
- New: apps/web/sentry.server.config.ts
- New: apps/web/sentry.edge.config.ts
- Modify: apps/web/next.config.js
- Modify: .env (add SENTRY_DSN)

Implementation notes:

- Sign up for Sentry free tier at sentry.io
- Get DSN, add to .env
- Use @sentry/nextjs with default config
- Mask PII in error context (don't log party names, amounts)

Tests:

- Throw a test error, confirm it appears in Sentry dashboard
- Verify PII is masked in the error context

Acceptance criteria:

- ☐ @sentry/nextjs installed and configured
- ☐ SENTRY_DSN in .env
- ☐ Throw an error in dev, see it in Sentry dashboard
- ☐ PII (party_name, amount) is scrubbed from error context

Sonnet prompt:

The user needs to provide a Sentry DSN. Either:

- (a) Ask them to sign up at sentry.io free tier and provide DSN
- (b) Set up with a placeholder DSN they can swap later

Install and configure Sentry for Next.js:

1. `pnpm add @sentry/nextjs --filter web`
 2. Run ``npx @sentry/wizard@latest -i nextjs`` OR manually create:
 - `apps/web/sentry.client.config.ts`
 - `apps/web/sentry.server.config.ts`
 - `apps/web/sentry.edge.config.ts`
 3. Wrap `next.config.js` with `withSentryConfig`
 4. Add scrub rules: redact `party_name`, `vendor_name`, `customer_name`, `amount` fields from error context
 5. Add to `.env.example`: `SENTRY_DSN=`
 6. Test by adding an intentional throw in a test API route, verify it appears in Sentry
-

Day 10 — LLM cost guardrails (verify + enforce)

Objective: Confirm `queryLimit` is enforced. Add hard daily cap per org.

Files:

- Audit: `apps/web/src/app/api/v1/query/route.ts`
- Audit: `apps/web/src/app/api/v1/llm-proxy/chat/route.ts`
- New if needed: `apps/web/src/lib/usage-guardrails.ts`

Implementation notes:

- Check if `queriesUsed >= queryLimit` blocks queries currently
- If not, add the check at the start of every LLM-calling endpoint
- Add daily cap: 100 queries/day per org regardless of plan (prevent runaway)
- Return 429 with friendly message: "Daily limit reached, contact support"

Tests:

- Test that 101st query in a day is blocked
- Test that `queryLimit` blocks at plan limit
- Test that resetting `queriesResetAt` clears the counter

Acceptance criteria:

- ☐ Daily hard cap enforced (100/day default)
- ☐ Plan limit enforced (`queryLimit` from Organisation)
- ☐ Resetting monthly (`queriesResetAt` cron job)
- ☐ Returns 429 with clear message

Sonnet prompt:

Read:

- apps/web/src/app/api/v1/query/route.ts
- apps/web/src/app/api/v1/llm-proxy/chat/route.ts
- packages/db/prisma/schema.prisma (Organisation.queriesUsed, queryLimit, queriesResetAt)

Audit and enforce LLM cost guardrails:

1. Check if queryLimit is currently enforced in the query route. If not, add it.
 2. Create apps/web/src/lib/usage-guardrails.ts:
 - checkOrgQueryAllowance(orgId): Promise<{ allowed: boolean; reason?: string; resetAt?: Date }>
 - incrementOrgQueryCount(orgId): Promise<void>
 - Both wrapped in a transaction to avoid races
 3. Add a DAILY_LIMIT constant (default 100/day per org) on top of monthly queryLimit
 4. Track daily via a new field (or use a Redis-like in-memory cache for first version)
 5. Call checkOrgQueryAllowance() at start of /api/v1/query and /api/v1/llm-proxy/chat
 6. Increment after successful response
 7. Return 429 with JSON: { error, dailyLimitReached, planLimitReached, resetAt }
 8. Write tests for both limits
-

Week 3: Authentication & Billing

Day 11 — Email service integration (Resend)

Objective: Centralised email sending via Resend. Templates for transactional emails.

Files:

- Modify: apps/web/src/lib/email.ts (currently stub)
- Add: apps/web/src/lib/email-templates/ (HTML templates)
- Add Resend API key to .env

Implementation notes:

- Sign up at resend.com (free tier: 100/day, 3K/month)
- Verify aiql.io domain (DNS records)
- Create base template + 3 specific: verify-email, reset-password, welcome
- Use React Email or simple HTML strings

Tests:

- Mock Resend client in tests
- Test that emails fire on signup, verification, password reset

Acceptance criteria:

- ☐ Resend account set up, domain verified
- ☐ `sendEmail()` function in `email.ts`
- ☐ 3 templates ready: `verifyEmail`, `resetPassword`, `welcome`
- ☐ Test email sends successfully in dev

Sonnet prompt:

The user needs to provide a Resend API key. If not provided, use a stub that logs to console.

Read:

– `apps/web/src/lib/email.ts` (current stub)

Build email infrastructure:

1. `pnpm add resend --filter web`
 2. Update `apps/web/src/lib/email.ts`:
 - export async function `sendEmail({ to, subject, html, from? }): Promise<void>`
 - Use Resend client when `RESEND_API_KEY` is set, else `console.log`
 3. Create `apps/web/src/lib/email-templates/`:
 - `base.ts` – wrapper layout (header, body, footer, AIQL branding)
 - `verify-email.ts` – verification link template
 - `reset-password.ts` – reset link template
 - `welcome.ts` – first-time user welcome
 4. Each template: takes data object, returns `{ subject, html }`
 5. Add `RESEND_API_KEY` to `.env.example`
 6. Add a test API route `/api/internal/test-email` (dev only) to send a test
 7. Test by running the dev server and hitting the test endpoint
-

Day 12 — Email verification on signup

Objective: New signups must verify email before they can use the app.

Files:

- Modify: `apps/web/src/app/api/auth/signup/route.ts`
- Modify: `packages/db/prisma/schema.prisma` (add `User.emailVerified`, `User.verificationToken`)
- New: `apps/web/src/app/api/auth/verify-email/route.ts`
- New: `apps/web/src/app/verify-email/page.tsx`

Implementation notes:

- Add `emailVerified: DateTime?` and `verificationToken: String?` to `User`
- On signup: create user with `emailVerified: null`, send verification email
- Verify endpoint: token → mark verified, clear token
- Login allowed but limited (banner: "Please verify your email")
- Hard-block after 7 days

Tests:

- Test signup creates user with unverified status
- Test verification endpoint marks user verified
- Test expired token rejected

Acceptance criteria:

- ☐ Signup sends verification email
- ☐ Click link → user verified
- ☐ Verification token expires after 7 days
- ☐ Unverified users see a banner

Sonnet prompt:

Read:

- apps/web/src/app/api/auth/signup/route.ts
- apps/web/src/lib/email.ts (from Day 11)
- packages/db/prisma/schema.prisma (User model)

Build email verification:

1. Update Prisma schema – add to User:
 - emailVerifiedAt: DateTime?
 - verificationToken: String?
 - verificationExpiresAt: DateTime?
 2. Run prisma generate (skip db push – use ALTER TABLE via direct SQL like we did before for the lastAppliedAt field)
 3. Update signup route:
 - Generate verification token (crypto.randomBytes(32).toString('hex'))
 - Set verificationExpiresAt = now + 7 days
 - Send verification email
 4. Create /api/auth/verify-email?token=xxx endpoint:
 - Find user by token
 - Check not expired
 - Set emailVerifiedAt, clear token
 5. Create /verify-email page that calls the endpoint
 6. Add banner to dashboard if !user.emailVerifiedAt
 7. Tests
-

Day 13 — Password reset flow

Objective: User can reset forgotten password via email link.

Files:

- New: apps/web/src/app/(auth)/forgot-password/page.tsx
- New: apps/web/src/app/(auth)/reset-password/page.tsx

- New: `apps/web/src/app/api/auth/forgot-password/route.ts`
- New: `apps/web/src/app/api/auth/reset-password/route.ts`
- Schema: add `User.resetToken`, `User.resetExpiresAt`

Implementation notes:

- `/forgot-password`: enter email → triggers reset email (always returns success to prevent email enumeration)
- `/reset-password?token=xxx`: form to set new password
- Token expires in 1 hour
- After reset: invalidate all existing sessions

Tests:

- Test request always returns success
- Test reset with valid token works
- Test reset with expired token rejected
- Test sessions invalidated after reset

Acceptance criteria:

- ☐ "Forgot password?" link on login page
- ☐ Email sent with reset link (or success returned silently if email doesn't exist)
- ☐ Reset page accepts new password
- ☐ Token expires in 1 hour
- ☐ All sessions invalidated after reset

Sonnet prompt:

Read:

- `apps/web/src/app/(auth)/login/page.tsx`
- `apps/web/src/app/api/auth/login/route.ts`
- `apps/web/src/lib/email.ts`

Build password reset flow:

1. Schema additions (use direct SQL ALTER):
 - `User.resetToken`: `String?`
 - `User.resetExpiresAt`: `DateTime?`
2. Create `/forgot-password` page with email form
3. Create `/api/auth/forgot-password`:
 - Look up user by email
 - Generate token, set 1-hour expiry
 - Send reset email
 - ALWAYS return 200 (no enumeration)
4. Create `/reset-password` page with token (query param) + new password form
5. Create `/api/auth/reset-password`:
 - Validate token
 - Hash new password (`argon2`)

- Update user
 - Clear token
 - Invalidate all sessions (`lucia.invalidateUserSessions`)
6. Add "Forgot password?" link to login page
 7. Create `reset-password.ts` email template (already started Day 11)
 8. Tests for all paths
-

Day 14 — Razorpay integration (account + checkout)

Objective: Customer can upgrade plan via Razorpay checkout. Webhook updates DB.

Files:

- New: `apps/web/src/lib/razorpay.ts`
- New: `apps/web/src/app/api/v1/billing/create-order/route.ts`
- New: `apps/web/src/app/api/v1/billing/webhook/route.ts`
- Modify: schema add Subscription model (or just Organisation fields)

Implementation notes:

- Sign up at razorpay.com (KYC required, takes 1-2 days for business)
- Use Razorpay Subscriptions (recurring) not one-time orders
- Plans created in Razorpay dashboard, IDs stored in env
- Webhook verifies signature, updates `Organisation.stripeSubscriptionId` (rename to `billingId`)

Tests:

- Mock Razorpay client
- Test webhook signature validation
- Test subscription state updates

Acceptance criteria:

- ☐ Razorpay account set up + plans created
- ☐ `/api/v1/billing/create-order` creates a Razorpay subscription
- ☐ Webhook receives + verifies + updates DB
- ☐ Settings page shows current plan + "Manage billing" link

Sonnet prompt:

The user needs to set up a Razorpay account first (requires KYC). For now, set up with test API keys.

Read:

- `packages/db/prisma/schema.prisma` (Organisation model – note `stripeCustomerId/stripeSubscriptionId` fields)

- apps/web/src/lib/plans.ts (from Day 5)

Build Razorpay subscription integration:

1. pnpm add razorpay --filter web
 2. Create apps/web/src/lib/razorpay.ts:
 - Razorpay client initialised with env keys
 - createSubscription(planId, customerInfo): Promise<{ subscriptionId, shortUrl }>
 - cancelSubscription(subscriptionId): Promise<void>
 - verifyWebhookSignature(body, signature, secret): boolean
 3. Create /api/v1/billing/create-order:
 - Takes { planTier: 'starter' | 'professional' | 'business' }
 - Auth required, gets org
 - Creates Razorpay subscription
 - Returns checkout URL
 4. Create /api/v1/billing/webhook:
 - Verify Razorpay signature
 - Handle events: subscription.activated, subscription.charged, subscription.cancelled
 - Update Organisation.stripeSubscriptionId (or rename to billingSubscriptionId)
 - Update plan field accordingly
 5. Add RAZORPAY_KEY_ID, RAZORPAY_KEY_SECRET, RAZORPAY_WEBHOOK_SECRET to .env.example
 6. Tests for webhook signature verification
-

Day 15 — Plan upgrade/downgrade UI

Objective: User can change plans from /settings/billing.

Files:

- New or modify: apps/web/src/app/(dashboard)/settings/billing/page.tsx
- New: apps/web/src/components/billing/plan-card.tsx
- New: apps/web/src/app/api/v1/billing/change-plan/route.ts

Implementation notes:

- Show current plan with usage metrics
- Show other plans with "Upgrade" / "Downgrade" buttons
- On change: cancel current Razorpay subscription, create new one
- Show pro-rated bill explanation

Tests:

- Test plan change cancels old, creates new
- Test downgrade preserves grandfather pricing

Acceptance criteria:

- ☐ /settings/billing shows current plan + usage
- ☐ Other plans displayed with clear upgrade/downgrade paths

- ☐ Click button → Razorpay checkout → webhook updates DB → success
- ☐ Confirmation dialog for downgrades

Sonnet prompt:

Read:

- apps/web/src/lib/plans.ts (from Day 5)
- apps/web/src/lib/razorpay.ts (from Day 14)
- apps/web/src/app/(dashboard)/settings/ (existing settings layout)

Build /settings/billing page:

1. If billing page exists, audit it. Else create.
 2. Create apps/web/src/components/billing/plan-card.tsx – single plan display with action button
 3. Page sections:
 - Current plan summary (name, price, billing date, usage: queries used/limit)
 - All available plans (3 cards) with Upgrade/Downgrade CTAs
 - Cancel subscription button (with confirmation)
 4. Create /api/v1/billing/change-plan:
 - Cancel current subscription
 - Create new at the new plan
 - Update DB
 5. Show pro-rated bill explanation
 6. Add tests
-

Week 4: Differentiator + Polish

Day 16 — TDS deduction scanner check (Section 40(a)(ia))

Objective: Add a scanner check that flags vendor payments above ₹30K with no TDS deducted.

Files:

- Modify: packages/close-engine/src/scanner.ts (add check function)
- Tests: packages/close-engine/src/__tests__/scanner.test.ts

Implementation notes:

- TDS thresholds for 194C (contractor): single payment ₹30K OR aggregate ₹1L/year
- Detection: find vendor payments where party_name exists, voucher_type = Payment, amount > threshold, and no corresponding TDS entry on the same voucher
- Output: list of vouchers + total ₹ exposure (which = the disallowed expense risk)

Tests:

- Mock SQL returns vendor payments > 30K
- Verify check identifies them as missing TDS

Acceptance criteria:

- ☐ New check function `checkTdsDeduction` added
- ☐ Returns Issue with code `tds_missing`
- ☐ Wired into `runDataQualityScan` parallel execution
- ☐ Test cases cover: single payment > 30K, aggregate > 1L, payment with TDS (should NOT flag)

Sonnet prompt:

Read:

- `packages/close-engine/src/scanner.ts`
- `packages/close-engine/src/__tests__/scanner.test.ts` (existing pattern for tests with mocked prisma)

Add TDS deduction scanner check (Section 40(a)(ia)):

1. Add function `checkTdsDeduction(table, start, end, colMap, presentColumns)` in `scanner.ts`:
 - SQL: find payment vouchers where `party_name` IS NOT NULL, `sum(debit_amount) > 30000` per voucher
 - Cross-check: no entry with `account_name` LIKE `'%TDS%'` in same voucher (reference_number match)
 - Return Issue: `code='tds_missing', severity='critical'`
 - title: "N vendor payments missing TDS deduction"
 - exposure = sum of payments (this is the disallowed expense risk)
 2. Add to `runDataQualityScan` parallel checks array
 3. Add issue title mapping in `task-generator.ts`:
`tds_missing: "Add TDS deduction for ${N} vendor payments"`
 4. Add tests in `scanner.test.ts`:
 - Mock returns 3 payments, each above 30K, no TDS
 - Assert `issue.affectedRows = 3`
 - Assert `issue.severity = 'critical'`
 5. Run `pnpm test`
-

Day 17 — Cash payment > ₹10K scanner check (Section 40A(3))

Objective: Flag cash payments above ₹10K — Income Tax Act disallows the expense.

Files:

- Modify: `packages/close-engine/src/scanner.ts`
- Tests: same file

Implementation notes:

- Detection: `voucher_type = Payment`, `account_name` LIKE `'%cash%'` (or Petty Cash etc.), `debit_amount > 10000`
- Section 40A(3) disallows cash payments > ₹10,000 single transaction (₹35K for transporters)

Tests:

- Mock cash payment > 10K → flag
- Mock cash payment <= 10K → don't flag

Acceptance criteria:

- ☐ Function `checkCashPayments` added
- ☐ Returns Issue with code `cash_payment_excess`
- ☐ Threshold configurable (default 10000)
- ☐ Tests pass

Sonnet prompt:

Read: `packages/close-engine/src/scanner.ts` (and the TDS check from Day 16 as reference)

Add Cash payment > ₹10K check (Section 40A(3)):

1. Function `checkCashPayments(...)`: looks for cash account payments above ₹10,000
 2. SQL: `LOWER(account_name) LIKE '%cash%' OR LIKE '%petty cash%', voucher_type='Payment', sum(debit_amount) per voucher > 10000`
 3. Issue: code='cash_payment_excess', severity='review', exposure = sum (disallowance risk)
 4. Title: "{\$N} cash payments above ₹10K – Section 40A(3) disallowance risk"
 5. Add to `runDataQualityScan` parallel checks
 6. Add tests
-

Day 18 — Cash receipt > ₹2L scanner check (Section 269ST)

Objective: Flag cash receipts above ₹2 lakh — penalty under Section 269ST.

Files:

- Modify: `packages/close-engine/src/scanner.ts`
- Tests: same file

Implementation notes:

- Detection: `voucher_type = Receipt, account_name LIKE '%cash%', credit_amount > 200000`
- Section 269ST: penalty = amount received in cash above ₹2L

Tests:

- Mock cash receipt > 2L → flag with severity critical (penalty equal to amount)

Acceptance criteria:

- ☐ Function `checkCashReceipts` added

- ☐ Returns Issue with code `cash_receipt_excess`
- ☐ Severity: critical (penalty equal to amount)
- ☐ Tests pass

Sonnet prompt:

Read: `packages/close-engine/src/scanner.ts` (and existing checks as reference)

Add Cash receipt > ₹2L check (Section 269ST):

1. Function `checkCashReceipts`: looks for cash account receipts above ₹2,00,000 single transaction
 2. SQL: similar to cash payment but Receipt type and `credit_amount` instead
 3. Issue: code='cash_receipt_excess', severity='critical', exposure = sum (penalty equals amount)
 4. Title: "\${N} cash receipts above ₹2L – Section 269ST penalty risk"
 5. Add to `runDataQualityScan`
 6. Tests
-

Day 19 — Account deletion / data export (DPDP compliance)

Objective: User can delete their account and export all their data.

Files:

- New: `apps/web/src/app/(dashboard)/settings/data/page.tsx`
- New: `apps/web/src/app/api/v1/account/export/route.ts`
- New: `apps/web/src/app/api/v1/account/delete/route.ts`

Implementation notes:

- Export: ZIP with org info, connections, scans, queries, knowledge entries — JSON files
- Delete: confirmation dialog (type org name), cascade delete (Cascade in schema handles most), drop upload tables

Tests:

- Test export generates valid ZIP
- Test delete removes all data

Acceptance criteria:

- ☐ `/settings/data` shows data overview + Export + Delete buttons
- ☐ Export downloads ZIP with all org data
- ☐ Delete requires typing org name + sends confirmation email + 24h grace period

Sonnet prompt:

Read:

- apps/web/src/app/(dashboard)/settings/ (existing pages)
- packages/db/prisma/schema.prisma (all org-related models)

Build DPDP-compliant account management:

1. Create /settings/data page with:
 - Data overview (X connections, Y MB stored, Z queries logged)
 - "Export all data" button
 - "Delete account" button (danger zone)
 2. Create /api/v1/account/export:
 - Gather: org info, all connections, all uploads, all scans, all queries, all knowledge
 - Anonymise tokens if needed
 - Generate ZIP using node:archiver
 - Stream response with proper headers
 3. Create /api/v1/account/delete:
 - Require org name typed exactly
 - Send email to all team members
 - Schedule deletion 24 hours later (immediate for now – add scheduling later)
 - Drop all upload_ tables
 - Delete Organisation (cascades to all related)
 4. Tests
-

Day 20 — Audit log for PII access

Objective: Log who in the org accessed which connection's data, when. Surface in settings.

Files:

- Schema: add ConnectionAccessLog model
- New: apps/web/src/lib/audit-log.ts
- Modify: connection-viewing pages to log
- New: apps/web/src/app/(dashboard)/settings/audit/page.tsx

Implementation notes:

- New table: connection access log (userId, connectionId, action, timestamp, ipAddress)
- Log on every page view of /connections/[id], scan, query
- Settings page lists last 100 entries with filters

Tests:

- Test that visiting a connection page creates a log entry
- Test that settings page renders logs

Acceptance criteria:

- ☐ Schema updated with ConnectionAccessLog

- ☐ Logging middleware/helper in place
- ☐ /settings/audit page shows recent access
- ☐ Filters: by user, by connection, by action

Sonnet prompt:

Read: packages/db/prisma/schema.prisma

Build PII access audit logging:

1. Schema (use ALTER TABLE direct SQL, not prisma db push):

```
ConnectionAccessLog {
  id          String    @id @default(cuid())
  orgId       String
  userId      String
  connectionId String?
  action       String    // 'view', 'scan', 'query', 'export'
  ipAddress   String?
  userAgent   String?
  metadata    String?    // JSON for action-specific data
  createdAt   DateTime @default(now())
  @@index([orgId, createdAt])
}
```

2. Create apps/web/src/lib/audit-log.ts:

- logAccess({ userId, orgId, connectionId, action, request }): Promise<void>
- Best-effort (don't fail the parent request)

3. Add log calls in:

- /connections/[id] page (action: 'view')
- /api/v1/connections/[connectionId]/scan (action: 'scan')
- /api/v1/query (action: 'query')
- export endpoints (action: 'export')

4. Create /settings/audit page:

- Table of last 100 entries
- Filters: user, connection, action, date range
- CSV export of full log

5. Tests

Phase 2 — Bank Reconciliation (Days 21-50)

Goal: Add the most-used CA workflow. Converts AIQL from monthly scanner to weekly tool.

Weeks 5-6: PDF Parsers

Day 21 — Bank statement schema design

Objective: Database tables for bank statements, transactions, match status.

Files:

- Schema: add BankStatement, BankTransaction, BankMatch
- Migration via direct SQL

Implementation notes:

- BankStatement: per-upload metadata (bank, account, period, file)
- BankTransaction: individual rows from statement (date, amount, type, description)
- BankMatch: links bank transaction to GL voucher (or 'unmatched')

Tests:

- Schema validates
- Basic CRUD works

Acceptance criteria:

- ☐ 3 new tables in schema
- ☐ Migrations applied
- ☐ Prisma client regenerated

Sonnet prompt:

Read: packages/db/prisma/schema.prisma

Design bank reconciliation schema. Add 3 models:

```
BankStatement {
  id          String @id @default(cuid())
  connectionId String
  bankName    String      // 'HDFC', 'ICICI', 'SBI', 'Axis', 'OTHER'
  accountNumber String    // last 4 digits stored, full encrypted
  periodStart DateTime
  periodEnd   DateTime
  openingBalance Decimal
  closingBalance Decimal
  originalFile String?    // S3 key
  uploadedAt  DateTime @default(now())
  transactions BankTransaction[]
}

BankTransaction {
  id          String @id @default(cuid())
  statementId String
  transactionDate DateTime
  description  String
  referenceNumber String? // cheque no, UTR, etc.
  amount       Decimal    // positive = credit, negative = debit
  balance      Decimal?
```

```

    matchedTo      BankMatch?
    @@index([statementId])
}

BankMatch {
  id              String @id @default(cuid())
  bankTransactionId String @unique
  voucherReference String? // GL reference_number
  matchConfidence Float    // 0-1
  matchType       String    // 'exact', 'fuzzy', 'manual', 'unmatched'
  inTransit       Boolean @default(false)
  note            String?
  matchedBy       String?
  matchedAt       DateTime?
}

```

Apply via ALTER TABLE direct SQL (similar to the OrgBusinessKnowledge migration pattern).
Run prisma generate.

Day 22-24 — HDFC PDF parser

Day 22 Objective: Extract transactions from a standard HDFC bank statement PDF.

Day 23 Objective: Handle HDFC PDF format variations.

Day 24 Objective: Tests + edge cases (multi-page, transactions split across pages).

Files:

- New: packages/bank-recon/src/parsers/hdfc.ts
- New: packages/bank-recon/src/parsers/types.ts
- Tests: packages/bank-recon/src/__tests__/hdfc.test.ts

Implementation notes:

- New monorepo package: packages/bank-recon
- Use pdf-parse to extract text, then regex/parsing rules
- HDFC has predictable format: Date | Narration | Chq/Ref | Value Date | Withdrawal | Deposit | Balance
- Need 5+ real HDFC statements to test against

Tests:

- Parse a known HDFC statement, assert specific transactions extracted
- Edge cases: multi-line narration, empty cells, footer pages

Acceptance criteria:

- ☐ Function `parseHdfcStatement(buffer): Promise<BankStatement>` returns structured data
- ☐ Handles 3+ format variations
- ☐ Tests with 5+ real (sanitised) HDFC PDFs

Sonnet prompt (Day 22):

Create new monorepo package `packages/bank-recon`. Mimic the structure of `packages/close-engine`.

Build HDFC bank statement PDF parser:

1. `pnpm add pdf-parse --filter @aiql/bank-recon`
2. Create `packages/bank-recon/src/parsers/types.ts`:
 - `ParsedStatement` type matching the `BankStatement` schema
 - `ParsedTransaction` type matching `BankTransaction` schema
3. Create `packages/bank-recon/src/parsers/hdfc.ts`:
 - export async function `parseHdfcStatement(buffer: Buffer): Promise<ParsedStatement>`
 - Use `pdf-parse` to extract text
 - Detect HDFC format: header contains "HDFC BANK", account number in standard place
 - Extract: account number, period, opening/closing balance
 - Parse transaction rows: Date | Narration | Chq No | Value Date | Withdrawal | Deposit | Balance
 - Convert to `ParsedTransaction[]`
4. Create `packages/bank-recon/package.json` with proper dependencies
5. Write tests `tests/__tests__/hdfc.test.ts` with a sample HDFC text dump (no real PDF in repo)
6. The user will provide a real HDFC PDF for manual verification

Day 23-24 follow on the same pattern — refine the parser, add format variations, comprehensive tests.

Day 25-26 — ICICI PDF parser

Same pattern as HDFC. ICICI format is similar but column order differs.

Sonnet prompt (Day 25):

Read:

- `packages/bank-recon/src/parsers/hdfc.ts` (built Day 22-24)
- `packages/bank-recon/src/parsers/types.ts`

Build ICICI bank statement parser following the HDFC pattern:

1. Create `packages/bank-recon/src/parsers/icici.ts`
 2. ICICI format: Date | Mode | Particulars | Deposits | Withdrawals | Balance
 3. Header marker: "ICICI BANK" + account number format
 4. Same return type `ParsedStatement`
 5. Tests in `packages/bank-recon/src/__tests__/icici.test.ts`
-

Day 27-28 — SBI PDF parser

Same pattern. SBI has the most format variations of major Indian banks.

Day 29 — Axis Bank PDF parser

Same pattern.

Day 30 — Generic CSV import + parser dispatcher

Objective: Fallback CSV parser + auto-detect which parser to use based on PDF header.

Files:

- New: `packages/bank-recon/src/parsers/csv.ts`
- New: `packages/bank-recon/src/parsers/dispatcher.ts`

Sonnet prompt:

Read all parsers in `packages/bank-recon/src/parsers/` (HDFC, ICICI, SBI, Axis)

Build:

1. Generic CSV parser:
 - Auto-detect columns using `@aiql/erp-connectors mapColumn` function
 - Map to `ParsedTransaction[]`
 2. Parser dispatcher:
 - Function `detectBankStatement(buffer, filename): { parser: 'hdfc' | 'icici' | 'sbi' | 'axis' | 'generic_csv', confidence: number }`
 - Read first page of PDF / first 5 lines of CSV
 - Look for bank-specific markers
 - Return appropriate parser
 3. Main entry: `parseBankStatement(buffer, filename, hint?): ParsedStatement`
 - Use dispatcher, then call appropriate parser
 - Fall back to CSV if PDF doesn't match known formats
 4. Tests for dispatcher with each bank format
-

Weeks 7-8: Matching Engine

Day 31 — Bank statement upload UI + API

Objective: User uploads a bank statement; AIQL parses, stores, shows summary.

Files:

- New: `apps/web/src/app/(dashboard)/connections/[id]/bank-recon/page.tsx`

- New: `apps/web/src/app/(dashboard)/connections/[id]/bank-recon/upload/page.tsx`
- New: `apps/web/src/app/api/v1/bank-recon/upload/route.ts`

Implementation notes:

- Upload page with drag-drop file
- POST file to `/api/v1/bank-recon/upload`
- Parse, store BankStatement + BankTransactions
- Redirect to `/bank-recon/{statementId}`

Sonnet prompt:

Read:

- `apps/web/src/app/(dashboard)/connections/[id]/page.tsx`
- `packages/bank-recon/*` (parsers built Days 22–30)

Build bank statement upload:

1. `/api/v1/bank-recon/upload` POST endpoint:
 - Accept multipart file
 - Determine `connectionId` from query/body
 - Use `bank-recon` dispatcher to detect format
 - Parse statement
 - Store BankStatement + BankTransactions in DB
 - Return `statementId`
 2. `/connections/[id]/bank-recon/upload` page:
 - Drag-drop file input
 - Bank format selector (or auto-detect)
 - Submit → call API → redirect to results
 3. `/connections/[id]/bank-recon` page:
 - List uploaded statements for this connection
 - "Upload new" CTA
 4. Tests for upload endpoint
-

Day 32 — Exact matching algorithm

Objective: Match bank transactions to GL vouchers where amount + date + ref number match exactly.

Files:

- New: `packages/bank-recon/src/matching/exact.ts`
- New: `apps/web/src/app/api/v1/bank-recon/[statementId]/match/route.ts`

Implementation notes:

- Query all GL vouchers in the bank statement's period
- For each bank transaction, look for: voucher with same amount, same date (or ± 1 day), same ref if available

- High-confidence exact matches (1.0)
- Store as BankMatch with type='exact'

Sonnet prompt:

Build exact matching engine:

1. Create packages/bank-recon/src/matching/exact.ts:
 - Function exactMatch(bankTransaction, glVouchers): MatchResult | null
 - Match criteria: same amount ($\pm ₹0.50$), same date (± 1 day), same reference if available
 - Returns { voucherReference, confidence: 1.0 } or null
 2. Create /api/v1/bank-recon/[statementId]/match POST endpoint:
 - Load all bank transactions for statement
 - Query GL vouchers in statement's period
 - For each bank transaction, attempt exact match
 - Store BankMatch records
 - Return summary
 3. Tests with mocked GL data
-

Day 33 — Fuzzy matching algorithm

Objective: Match remaining (unmatched-after-exact) transactions using fuzzy criteria.

Files:

- New: packages/bank-recon/src/matching/fuzzy.ts

Implementation notes:

- For unmatched bank transactions, look for:
 - Same amount ($\pm ₹1$), date within ± 7 days, description contains vendor name
 - Or description similar to voucher narration (Levenshtein distance)
- Confidence based on closeness: 0.7-0.95
- Store with type='fuzzy'

Sonnet prompt:

Build fuzzy matching:

1. packages/bank-recon/src/matching/fuzzy.ts:
 - fuzzyMatch(bankTransaction, glVouchers): MatchResult | null
 - Scoring: amount weight 50%, date weight 30%, description similarity 20%
 - Threshold: confidence ≥ 0.7 to suggest a match
 2. Integrate into match endpoint from Day 32 (run fuzzy after exact for remaining)
 3. Tests
-

Day 34 — Match scoring + confidence levels

Objective: Combine exact + fuzzy results, surface confidence for user review.

Sonnet prompt:

Refine matching engine:

1. Combine exact + fuzzy results
 2. Add confidence buckets:
 - 'high' (0.95–1.0): auto-confirm, show as matched
 - 'medium' (0.7–0.94): show as "review suggested"
 - 'low' (<0.7): show as unmatched
 3. Update BankMatch records with confidence bucket
 4. Tests
-

Day 35 — Match results storage + retrieval

Sonnet prompt:

Build match results API:

1. GET /api/v1/bank-recon/[statementId]/matches:
 - Returns categorized: matched (exact), suggested (fuzzy), unmatched (bank side), unmatched (book side)
 - Pagination for large statements
 2. GET /api/v1/bank-recon/[statementId] returns statement summary + match counts
 3. Tests
-

Day 36-37 — Manual match UI + persistence

Sonnet prompt (Day 36):

Build manual match UI:

1. /connections/[id]/bank-recon/[statementId] page:
 - Three columns: Matched | Suggested | Unmatched
 - Each transaction shows date, description, amount
 - Suggested matches show confidence and proposed voucher
 - Click "Confirm" / "Reject" / "Match to..."
 2. "Match to..." opens modal with searchable list of unmatched GL vouchers
 3. Drag-drop variant (post-MVP)
 4. Tests
-

Day 38-39 — In-transit tracking + carry-forward

Sonnet prompt (Day 38):

Add in-transit tracking:

1. UI option to mark unmatched transaction as "in transit" (cheque issued but not

- cleared)
 - 2. Updates BankMatch.inTransit = true
 - 3. Next month: these transactions appear in "carried forward" section
 - 4. Tests
-

Day 40 — Tests for matching engine

Sonnet prompt:

- Add comprehensive integration tests for bank-recon:
- End-to-end: upload statement → match → store → retrieve
 - Edge cases: zero transactions, all match, none match
 - Performance: 10K transactions match in < 30 seconds
-

Weeks 9-10: Workflow Integration

Day 41-42 — Bank Recon dashboard + discrepancy report PDF

Day 41:

- Build Bank Recon dashboard:
- Summary cards: matched count, unmatched bank, unmatched book, variance ₹
 - Three-bucket detail view (clickable)
 - "Export discrepancy report" button

Day 42:

- Reuse pdf-export pattern from Day 6:
- Generate discrepancy report PDF
 - Include all unmatched transactions
 - Recommended actions
-

Day 43-44 — Recurring patterns detection

- Detect recurring patterns:
- "Bank charges always match – auto-confirm next time"
 - "Same vendor pays monthly with similar reference – high confidence match"
 - Reduce review time on subsequent statements
-

Day 45 — Bank Recon as close period task

Integrate into Close Manager:

- Generate "Reconcile bank account" task during close period creation if BANK accounts exist
 - Task links to bank-recon page
 - Mark complete when all transactions matched or marked
-

Days 46-50 — Multi-bank support, polish, tests, launch

Day 46: Multi-bank per entity support **Day 47:** End-to-end tests **Day 48:** UI polish **Day 49:** Documentation **Day 50:** Phase 2 launch + pricing announcement email

Phase 3 — GSTN Auto-Reconciliation (Days 51-90)

Goal: Auto-pull GSTR-2A/2B from GST portal, reconcile against books. Major pricing justifier.

Prerequisites:

- GSTN sandbox API access (apply Week 1 of Phase 1 — 6-week approval)
 - GSP partnership decided (or direct integration)
-

Weeks 11-12: GSTN API Integration

Days 51-55 — GSTN authentication

Day 51: GSTN sandbox status, document API contract, set up env vars **Day 52:** Authentication flow — GSTIN registration **Day 53:** OTP flow handling (UI + API) **Day 54:** Session management + auth token storage **Day 55:** Token refresh logic

Sonnet prompt (Day 52):

GSTN API requires GSP (GST Suvidha Provider) credentials. The user must have these set up.

Read GSTN public API documentation: <https://docs.gstn.org/>

Build GSTN authentication module:

1. Create packages/gstn-integration/ monorepo package
2. Authentication flow:
 - User enters GSTIN
 - Backend calls GSTN auth API

- GSTN sends OTP to registered mobile/email
 - User enters OTP
 - Backend exchanges for access token
3. Store encrypted tokens per organisation (use existing crypto infra)
 4. Token TTL handling (refresh before expiry)
 5. Tests with mocked GSTN responses
-

Days 56-60 — Rate limiting, error handling, testing, encryption, UI

Day 56: Rate limiting (GSTN throttles aggressively) + retry logic with backoff **Day 57:**

Comprehensive error handling for GSTN-specific errors **Day 58:** Sandbox testing scenarios

(success, OTP fail, throttle, expired token) **Day 59:** GSTN connection UI (Settings → Integrations →

GSTN) **Day 60:** Encrypted credential storage

Weeks 13-14: GSTR-2A/2B Pull

Days 61-65 — GSTR-2A + 2B pull and parse

Day 61: GSTR-2A pull endpoint **Day 62:** GSTR-2A JSON parsing **Day 63:** GSTR-2A storage schema

(GstrPullRecord, GstrInvoice) **Day 64:** GSTR-2B pull endpoint **Day 65:** GSTR-2B JSON parsing

Days 66-70 — Schema, scheduler, aggregation, status tracking, tests

Day 66: GSTR-2B storage + invoice-level records **Day 67:** Monthly pull scheduler (cron or queue-

based) **Day 68:** Vendor-wise aggregation queries **Day 69:** Supplier filing status tracking **Day 70:**

Tests for full pull pipeline

Weeks 15-16: Reconciliation Engine

Days 71-75 — Comparison logic

Day 71: Books ITC vs 2B ITC comparison engine **Day 72:** Invoice-level matching algorithm (invoice

no + amount + date + GSTIN) **Day 73:** Match scoring for invoices **Day 74:** Rule 36(4) compliance

check (provisional ITC limit) **Day 75:** Provisional ITC tracking

Days 76-80 — Differential analysis, scorecard, UI, tests

Day 76: Find invoices in 2B not in books **Day 77:** Find invoices in books not in 2B **Day 78:** Vendor

scorecard (% filed on time, historical) **Day 79:** GST Recon dashboard UI **Day 80:** Tests for

Weeks 17-18: Workflow Integration

Days 81-85 — GSTR-3B prep, follow-up workflow

Day 81: GSTR-3B preparation helper (eligible ITC summary) **Day 82:** Eligible vs ineligible ITC categorisation **Day 83:** Email vendor about late filings — template + API **Day 84:** Follow-up workflow UI (track status of vendor follow-ups) **Day 85:** ITC at-risk dashboard (CFO view)

Days 86-90 — Trends, integration, launch

Day 86: Historical trend visualization (your ITC compliance over time) **Day 87:** Integration with TDS check (cross-reference vendor data) **Day 88:** GST Recon as close period task **Day 89:** End-to-end GST recon tests **Day 90:** Phase 3 launch + customer email + pricing announcement

Phase 4 — Tally Live Sync (Days 91-130)

Goal: Desktop agent that auto-pulls Tally data. Moves AIQL from monthly to daily product.

Prerequisites:

- Tally TDL expertise (you learn it Weeks 17-18, or hire specialist)
 - Code signing certificates (Windows + Mac) — ~₹15K + ₹10K
-

Weeks 19-21: Desktop Agent

Days 91-100 — Agent foundation

Day 91: Agent architecture design (Tauri vs Electron decision) **Day 92:** Tauri setup + cross-platform skeleton **Day 93:** Windows agent skeleton (system tray, settings, status) **Day 94:** Mac agent skeleton (menu bar, settings, status) **Day 95:** TDL/XML data extraction from Tally HTTP endpoint **Day 96:** Voucher extraction **Day 97:** Master data extraction (vendors, customers, accounts, GSTINs) **Day 98:** Delta detection logic (last sync timestamp, change tracking) **Day 99:** Local SQLite cache for sync state **Day 100:** Local PII tokenisation (agent strips PII before cloud upload)

Days 101-105 — Cloud sync, installer

Day 101: Tokenisation key management (key generation, secure storage in OS keychain) **Day 102:** Cloud sync endpoint (receive deltas from agent) **Day 103:** Sync state reconciliation (handle conflicts) **Day 104:** Windows installer (MSI build pipeline) **Day 105:** Mac installer (pkg build pipeline)

Weeks 22-23: Sync Pipeline

Days 106-115 — Scheduling, resilience, status

Day 106: Scheduled sync (cron-based daily at user time) **Day 107:** On-demand sync trigger (from agent menu) **Day 108:** Conflict detection logic **Day 109:** Conflict resolution UI (manual review) **Day 110:** Network resilience — retry with backoff **Day 111:** Network resilience — offline mode (queue locally) **Day 112:** Sync status dashboard (cloud-side view of agent health) **Day 113:** Sync error reporting + alerting **Day 114:** Agent metrics + telemetry (privacy-preserving) **Day 115:** Tests for sync pipeline

Weeks 24-25: Real-time Workflow

Days 116-125 — Notifications, alerts, integration

Day 116: Auto-trigger scan on new data sync **Day 117:** Notification system (email + in-app) **Day 118:** Critical finding alerts (immediate) **Day 119:** WhatsApp digest integration (via Gupshup or Twilio India) **Day 120:** Anomaly alerts — configurable thresholds **Day 121:** Alert subscription management (per user, per type) **Day 122:** Slack integration (optional) **Day 123:** Email digest builder (daily/weekly) **Day 124:** Tests for notification system **Day 125:** Audit trail of syncs

Week 26: Polish + Launch

Days 126-130

Day 126: Windows code signing (EV certificate) **Day 127:** Mac notarization (Apple Developer ID) **Day 128:** Auto-update mechanism (electron-updater or Tauri equivalent) **Day 129:** Agent documentation for IT teams (installation guide, firewall rules) **Day 130:** Phase 4 launch + customer email + pricing announcement + celebration

Templates & References

Sonnet prompt template (use for any day)

[Context section]

Read these files first to understand the codebase:

– {specific file paths from the day's entry}

[Task section]

[Copy the day's implementation notes + acceptance criteria]

[Verification section]

Before declaring done:

1. Run `pnpm typecheck`
2. Run `pnpm test` (or `pnpm --filter {package} test` for scoped)
3. Manually verify the acceptance criteria
4. Update `CLAUDE.md` if architecture changed

[Constraints]

- Use existing patterns and components – don't introduce new conventions
- Add new code in the appropriate package – don't put backend logic in `apps/web`
- Follow the existing test patterns (vi.hoisted for mocks, etc.)
- All tests must pass before marking complete

Weekly review checklist

Every Friday afternoon (30 minutes):

- ☐ What shipped this week vs plan?
- ☐ Any spillover to next week?
- ☐ Tests still all passing?
- ☐ Production stable (Sentry dashboard)?
- ☐ Any customer feedback to incorporate?
- ☐ Next week's day-1 prompt ready in advance?

When to deviate from the plan

Acceptable reasons to skip/swap a day:

- Customer urgently needs a feature not on the plan (bug fix, integration request)
- A planned feature reveals a deeper architectural issue that needs addressing first
- Phase 1 features need to ship faster than planned for an upcoming demo

Not acceptable reasons:

- "I don't feel like building X today"
- "Y looks more interesting"

- "Let me build the agent workflow instead"

The plan succeeds only if executed in order. Variations compound — by month 3, off-plan work means you've shipped a different product than designed.

Tracking sheet template

Maintain a simple Google Sheet:

Day	Date	Title	Status	Hours	Notes
1	2026-06-02	Demo loader UI	Done	4	All criteria met
2	2026-06-03	Onboarding step 1	In progress	6	Spillover 1 day

...

Update daily. Review weekly. Plan monthly.

Final Notes

What this plan delivers

After 130 days:

- AIQL is feature-complete for Indian SME finance teams + CA firms
- Pricing genuinely defensible at ₹2,499 / ₹5,999 / ₹11,999
- 30-60 paying customers (parallel customer acquisition runs)
- Real moat: workflow integration (GSTN, Tally, Bank Recon) + knowledge base data

What it doesn't deliver

These come LATER (Year 2):

- ISO 27001 / SOC 2 certifications
- Mobile native app
- Enterprise features (multi-org, BYOK encryption, dedicated support)
- International market features
- Banking / lending verticals

The critical reminder

This is a 6-month engineering plan. **It doesn't include customer acquisition work** — which is at least equal in time and importance. Use the separate Sales Playbook (AIQL_SALES_PLAYBOOK.pdf) for that.

Engineering without customers = expensive prototype. Customers without features = unsustainable promises. Both must happen in parallel.

Plan generated May 30, 2026. Revise quarterly based on actual progress and customer feedback.